

```
// Code-A3 (Fractical Knapsack)
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
// Structure for an item
```

```
struct Item {
```

```
    int weight;
```

```
    int value;
```

```
};
```

```
// Comparison function to sort items by value/weight ratio
```

```
bool compare(Item a, Item b) {
```

```
    double r1 = (double)a.value / a.weight;
```

```
    double r2 = (double)b.value / b.weight;
```

```
    return r1 > r2; // Sort in descending order
```

```
}
```

```
double fractionalKnapsack(int W, vector<Item>& items) {
```

```
    // Sort items by value-to-weight ratio
```

```
    sort(items.begin(), items.end(), compare);
```

```
    double totalValue = 0.0; // Total value in knapsack
```

```
    for (auto &i : items) {
```

```
        if (W == 0) break; // Knapsack is full
```

```
        // If item can be fully taken
```

```
        if (i.weight <= W) {
```

```
            W -= i.weight;
```

```

        totalValue += i.value;
    } else {
        // Take fraction of item
        totalValue += i.value * ((double)W / i.weight);
        W = 0;
    }
}
return totalValue;
}

```

```

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;

    vector<Item> items(n);
    cout << "Enter value and weight of each item:\n";
    for (int i = 0; i < n; i++)
        cin >> items[i].value >> items[i].weight;

    cout << "Enter capacity of knapsack: ";
    cin >> W;

    double maxValue = fractionalKnapsack(W, items);
    cout << "\nMaximum value in knapsack = " << maxValue << endl;

    return 0;
}

```

// SAMPLE OUTPUT

/*

* \$./a.out

* Enter number of items: 3

* Enter value and weight of each item:

* 12 32

* 56 67

* 23 87

* Enter capacity of knapsack: 45

*

* Maximum value in knapsack = 37.6119

*/